

# TorUsNet 7.0 ENG

L'Architech (Gabriel Breton-Breault)

## 1 General Introduction

### 1.1 — Foreword: Natural Evolution Toward the Name TorUsNet

The protocol initially named *TorusNet* evolves in version 7.0 toward a more natural identity: **TorUsNet**. This change reflects an important transition in the philosophy of the project.

In its early versions, TorusNet was designed as an independent directional protocol, demonstrating the feasibility of a communication architecture based on an inviolable private torus and an opaque public torus. With **TorUsNet 7.0**, the protocol reaches a decisive milestone: it is now intended to integrate naturally into a Tor browser, as a **directional transport layer** replacing classical mechanisms based on TCP and TLS.

The name *TorUsNet* expresses this dual nature:

- **Tor**: seamless integration into the Tor browser ecosystem.
- **Us**: continuity with the geometry of the torus, the core of directional security.
- **Net**: the primary function of the protocol, a non-linear and inviolable network layer.

TorUsNet is not a fork of Tor. It is a **directional architecture** designed to replace the transport layer while preserving the spirit of the Tor browser: anonymity, confidentiality, resilience, and freedom of access.

Version 7.0 marks the beginning of a convergence: **TorUsNet becomes a protocol designed for Tor, before being extended to the rest of the network.**

### 1.2 — Introduction of New Directional Key Types (transmitted over the network)

TorUsNet 7.0 introduces a new type of directional key, designed to operate exclusively with the model in which the directional path is transmitted over the network. These keys are sent *before* the path and serve to correctly initialize the private torus prior to reconstruction.

#### Structure of the new keys

Each directional key has the following structure:

- **20 bits — Starting point** Indicates the initial cell in the private torus where interpretation of the path begins.

- **16 bits — Cell ID** Identifies the targeted cell in the private torus for this key.

### Key summary

**Directional Key = 20 bits (start) + 16 bits (ID) = 36 bits**

These keys are compact, fast to transmit, and sufficient to initialize the private torus without exposing its internal geometry.

### Role in the protocol

These keys:

- prepare the private torus to receive the full path,
- define the initial directional zone,
- ensure coherence between the private torus and the transmitted path,
- enable more precise reconstruction,
- guarantee that the private torus interprets movements correctly.

They are transmitted *before* the path because they serve as directional pre-configuration.

### 1.3 — Direct Transmission of Directional Paths

In previous versions, the protocol used a seed to generate the directional path.

TorUsNet 7.0 introduces an alternative: **direct transmission of the complete directional path**.

From now on:

- the directional path can be sent directly over the network,
- the private torus uses it as-is to reconstruct the file,
- no internal information from the torus is exposed,
- security remains entirely within the internal geometry.

The transmitted path contains:

- no internal key,
- no private torus structure,
- no directional geometry,
- no exploitable information.

It is purely a **movement vector**, with no cryptographic value.

#### 1.4 — Transmission of the Reconstructed File Hash

To ensure that the file reconstructed by the private torus is:

- complete,
- intact,
- uncorrupted,
- coherent with the transmitted path,

TorUsNet 7.0 introduces the transmission of the **hash of the reconstructed file**.

This hash:

- is sent over the network,
- reveals no directional information,
- allows no derivation of the torus,
- compromises no key,
- and serves only to validate the integrity of the final file.

It becomes an external proof that directional reconstruction was performed correctly, even in cases of:

- key loss,
- partially received paths,
- multiple reconstructions,
- network interruptions.

#### 1.5 — Coexistence of Two Models: Seed or Transmitted Path

TorUsNet 7.0 now allows the seed to be completely replaced by a full path transmitted directly over the network. However, this direct transmission slightly increases network traffic.

For this reason, TorUsNet 7.0 maintains **both models**:

##### Model A — Directional Seed (compact, local)

- A 70-bit seed is transmitted.

- The private torus generates the path locally.
- Network traffic is minimal.
- Ideal for low-bandwidth environments.

### **Model B — Transmitted Directional Path (complete, external)**

- The full path is sent over the network.
- The seed is no longer required.
- The protocol becomes simpler to implement.
- Ideal for integration into a Tor browser.

### **Model selection**

The choice between seed or transmitted path must be made at implementation time, depending on:

- system constraints,
- available bandwidth,
- desired simplicity,
- application type,
- developer preference.

Both models are officially supported and documented in **TorUsNet 7.0**.

## **Chapter 1 — Introduction to the Directional Architecture (TorUsNet 7.0)**

### **1.1. General Vision**

The directional architecture is an entirely new communication technology. It does not rely on mathematics, cryptographic derivations, or classical structures, but on a living internal mechanism—impossible to derive, impossible to explore, impossible to falsify.

TorUsNet 7.0 is built upon four fundamental elements:

1. **The Private Torus** — an internal structure of 1,000,000 cells, each containing:
  - 1 normal bit,
  - 1 implicit phantom bit (NOT of the normal bit),

- 4 possible directions. The private torus is unique to each session, never transmitted, never exported.
2. **The Directional Mechanics** — a set of rules defining:
- how a key moves inside the torus,
  - how IDs mutate,
  - how internal paths activate,
  - how normal/phantom pairs invert,
  - how internal coherence is maintained.
3. **The Glass Fortress** — an internal software module that:
- validates directional coherence,
  - monitors directional ports,
  - monitors internal IDs,
  - destroys the session in case of anomaly,
  - blocks hostile IPs.
4. **Directional Keys Transmitted (TorUsNet 7.0)** — a new type of key sent over the network before the path:
- 20 bits: starting point,
  - 16 bits: cell ID, for a total of 36 bits per key. These keys prepare the private torus to receive the transmitted path.

The directional architecture does **not** use:

- encryption,
- mathematical derivation,
- signatures,
- certificates,
- private keys,
- cryptographic primitives.

It relies exclusively on:

- directional windows,
- directional constants,
- internal paths,
- ID mutations,
- internal routes,
- normal/phantom pairs,
- non-derivable internal structures.

## 1.2. Why a Directional Architecture?

Classical protocols (TLS, SSH, IPSec, WireGuard, etc.) rely on:

- mathematics,
- derivations,
- private keys,
- certificates,
- signatures.

These systems are vulnerable to:

- mathematical analysis,
- quantum computers,
- photonic computers,
- derivation attacks,
- collisions,
- replay attacks,
- injections.

The directional architecture eliminates these attack vectors entirely. It does not rely on any exploitable mathematical structure. It relies on an internal mechanism impossible to derive, impossible to explore, impossible to reverse.

Even with infinite computational power, an attacker cannot:

- derive a private torus,

- explore a private torus,
- reconstruct a private torus,
- predict a directional path,
- falsify a directional key,
- repeat a directional key,
- reconstruct an internal ID.

### **1.3. The Three Fundamental Elements (Version 7.0)**

#### **1.3.1. The Private Torus**

The Private Torus is an internal structure of 1,000,000 cells, each containing:

- 1 normal bit,
- 1 implicit phantom bit,
- 4 possible directions.

It is:

- unique to each session,
- fractally reconstructed,
- never transmitted,
- never exported,
- never derivable.

It contains:

- internal IDs,
- the global ID,
- the unique torus ID,
- internal paths,
- normal/phantom pairs.

**Inversion of normal/phantom bits during the path**

In TorUsNet 7.0, each time a cell is traversed by the directional path, the normal and phantom bits automatically invert:

- normal = 0  $\rightarrow$  1
- phantom = 1  $\rightarrow$  0 or vice versa.

This inversion:

- creates directional oscillation,
- modifies the internal geometry of the torus,
- prevents any prediction,
- reinforces non-derivability,
- makes each path unique and non-reproducible.

This rule is fundamental in TorUsNet 7.0.

### **1.3.2. The Directional Mechanics**

It defines:

- how a key moves inside the torus,
- how IDs mutate,
- how internal paths activate,
- how normal/phantom pairs invert,
- how internal coherence is maintained.

It is:

- non-linear,
- non-derivable,
- non-reversible,
- living,
- oscillating,
- mutating.

### **1.3.3. The Glass Fortress**



The Glass Fortress:

- validates directional coherence,
- monitors directional ports,
- monitors internal IDs,
- destroys the session in case of anomaly,
- blocks hostile IPs.

It is the directional equivalent of a structural firewall.

#### **1.4. Directional Ports: Extended Digital Ports**

Directional ports are:

- classical digital ports,
- but with an extended range in the billions,
- generated by directional keys,
- validated by the Glass Fortress.

They serve to:

- transport directional transmissions,
- avoid collisions,
- prevent scans,
- prevent injections,
- prevent flow hijacking.

Examples:

- 1,482,993,112
- 7,000,221,991
- 3,991,000,000

Impossible to scan. Impossible to predict. Impossible to falsify.

#### **1.5. A Post-Internet Architecture**

The directional architecture is designed for the era:

- post-quantum,
- post-cryptographic,
- photonic.

It does not depend on:

- mathematics,
- certificates,
- private keys,
- cryptographic primitives.

It still uses:

- DNS,
- HTTPS,

but only as a temporary entry gate before the session becomes **100% directional**.

### **1.6. A Directional Session: Living and Autonomous**

A directional session is:

- living,
- autonomous,
- self-coherent,
- self-validating,
- self-mutating.

It evolves with each key:

- ID mutation,
- normal/phantom inversion,
- internal progression,
- private torus update,
- validation by the Glass Fortress.

It cannot be:

- copied,
- cloned,
- derived,
- predicted,
- falsified.

### **1.7. Objectives of the Directional Protocol**

TorUsNet aims to:

1. Create sessions inviolable by design.
2. Replace classical cryptographic protocols.
3. Create directional private networks (multi-torus).
4. Enable directional reconstruction of files.
5. Create an internal communication layer (directional ports).
6. Create a unique directional identity.
7. Ensure post-quantum and photonic security.

### **1.8. Summary of Chapter 1**

The directional architecture:

- is not cryptographic,
- is not mathematical,
- is not derivable,
- is not falsifiable.

It relies on:

- a private torus,
- an internal mechanism,
- directional coherence,
- extended digital ports,
- a software Glass Fortress,

- transmitted directional keys,
- a transmitted or seed-generated path,
- a final hash validating reconstruction,
- automatic normal/phantom inversion during the path.

It represents a new generation of protocol, designed for the post-quantum and photonic era.

## **Chapter 2 — Redesign of the Directional Seed and Introduction of the Transmitted-Path Model (TorUsNet 7.0)**

TorUsNet 7.0 introduces a major evolution in the way directional trajectories are generated, transmitted, and interpreted. Two models now coexist:

- the classical model based on the **directional seed (70 bits)**
- the modern model based on the **path transmitted directly over the network**

Both models are officially supported and documented.

### **2.1 — Structure of the Directional Key (seed model, 128 bits)**

The full directional key retains its original 128-bit structure, composed of:

- **2 bits — Key type**
- **20 bits — Directional starting point**
- **20 bits — Directional ending point**
- **16 bits — Target cell ID**
- **70 bits — Directional seed**

**Total: 128 bits**

### **2.2 — Structure of the Directional Seed (70 bits)**

The directional seed is a compact 70-bit block containing five components:

- **Exponential base (4 bits)**
- **Fixed-tower selection (4 bits)**
- **Fixed-tower height (2 bits)**

- **Dynamic stages (48 bits)**
- **Final modulation (12 bits)**
  - Addition: 4 bits
  - Multiplication: 8 bits

### 2.2.1 — Exponential Base (4 bits)

The 4 bits define a base among 16 hierarchical values:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 100, 1000, 10,000, 100,000, 1,000,000, 10,000,000

### 2.2.2 — Fixed-Tower Selection (4 bits)

#### Complete list of the 15 available fixed towers

The fixed towers follow their expansion logic up to 15 possibilities, each following the rule:

$$n^{(n+1)(n+2)(n+3)} \text{ where } n \in [1, 15].$$

Complete list:

- Fixed Tower 1:  $1^{2^{3^4}}$
- Fixed Tower 2:  $2^{3^{4^5}}$
- Fixed Tower 3:  $3^{4^{5^6}}$
- Fixed Tower 4:  $4^{5^{6^7}}$
- Fixed Tower 5:  $5^{6^{7^8}}$
- Fixed Tower 6:  $6^{7^{8^9}}$
- Fixed Tower 7:  $7^{8^{9^{10}}}$
- Fixed Tower 8:  $8^{9^{10^{11}}}$
- Fixed Tower 9:  $9^{10^{11^{12}}}$
- Fixed Tower 10:  $10^{11^{12^{13}}}$
- Fixed Tower 11:  $11^{12^{13^{14}}}$

- Fixed Tower 12:  $12^{13^{14^{15}}}$
- Fixed Tower 13:  $13^{14^{15^{16}}}$
- Fixed Tower 14:  $14^{15^{16^{17}}}$
- Fixed Tower 15:  $15^{16^{17^{18}}}$

#### Code 0000: no fixed tower used

The 4 bits therefore allow:

- **0000** → no fixed tower
- **0001 to 1111** → selection of one fixed tower among the 15 possibilities

#### 2.2.3 — Fixed-Tower Height (2 bits)

Determines the number of levels used:

- **00** → 1 level
- **01** → 2 levels
- **10** → 3 levels
- **11** → 4 levels

#### 2.2.4 — Dynamic Stages (48 bits)

The 48 bits are divided into **6 dynamic stages**, each represented by an 8-bit raw value:

$$D_1, D_2, D_3, D_4, D_5, D_6 \in \{0, 1, \dots, 255\}$$

#### Directional rule:

- if a dynamic stage equals **0**, it is **ignored**,
- if its value is **> 0**, it is **active** in the exponential tower.

#### 2.2.5 — Final Modulation (12 bits)

The final modulation consists of:

- **Addition (A)** — 4 bits → 0 to 15
- **Multiplication (M)** — 8 bits → 0 to 255

The final exponential value becomes:

$$E_{\text{final}} = (X + A) \times M$$

where **X** represents the complete tower:

$$X = \text{Base}^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}$$

## 2.3 — Final Exponential Structure

With fixed tower:

$$\text{Base}^{e_1^{e_2^{e_3^{e_4^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}}}}}$$

Without fixed tower:

$$\text{Base}^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}$$

Then modulation:

$$E_{\text{final}} = (X + A) \times M$$

## 2.4 — Representation Capacity

- **Raw space:**

$$2^{48}$$

- **Exponential space:** far exceeds cryptographic, quantum, and photonic capabilities.

## 2.5 — Impact on the Private Torus

The private torus uses:

- the exponential base,
- the fixed tower (if activated),
- the 6 dynamic stages,
- the final modulation (A and M),
- the rule ignoring zero-value stages,

to determine the internal directional trajectory.

## **Normal/phantom inversion**

During execution of the path:

- each traversed cell inverts its normal and phantom bits,
- modifying the internal geometry of the torus,
- making each path unique and non-reproducible.

## **2.6 — Introduction of the Transmitted-Path Model (TorUsNet 7.0)**

TorUsNet 7.0 introduces a second model:

**The complete directional path is transmitted directly over the network.**

This model:

- removes the use of the seed,
- simplifies directional generation,
- enables natural integration into a Tor browser,
- slightly increases network traffic,
- requires the new 36-bit directional keys.

### **Transmission order:**

1. Directional keys (36 bits)
2. Complete directional path
3. Hash of the reconstructed file

## **2.7 — Conclusion of Chapter 2**

TorUsNet 7.0 introduces:

- a 70-bit directional seed,
- a multi-stage exponential engine,
- 15 fixed towers,
- configurable height,
- 6 dynamic stages (48 bits),
- a 12-bit final modulation (4-bit addition + 8-bit multiplication),



- a model based on transmitted paths,
- new 36-bit directional keys,
- transmission of the final hash,
- automatic normal/phantom inversion.

This structure reinforces the inviolable, non-derivable, and geometrically unpredictable nature of TorUsNet.

### **Chapter 3 — Glass Fortress (Structural Directional Firewall, TorUsNet 7.0)**

The Glass Fortress is the structural directional firewall of the private torus. It validates, monitors, and protects all directional operations according to the model chosen by the developers:

- **Model A:** Transmitted directional key (36 bits)
- **Model B:** Full directional key (128 bits)

Both models are supported, but must **never** be used simultaneously. The Glass Fortress automatically adapts to the active model.

#### **3.1. Overview**

The Glass Fortress validates:

- the directional key (36 bits or 128 bits depending on the model),
- the transmitted ID (16 bits),
- the reconstructed ID (16 bits),
- the fractal coherence of the torus,
- the directional reconstruction of the file,
- the normal/phantom inversion during the path.

It protects:

- the private torus,
- the directional key of the active model,
- directional coherence,

- file reconstruction.

It destroys the session in case of anomaly.

### **3.2. Fundamental Role**

**✓ Validation ✓ Monitoring ✓ Protection ✓ Destruction**

### **3.3. Joint Validation: Glass Fortress + Host**

Regardless of the chosen model (36 bits or 128 bits), the Glass Fortress always validates the directional key together with the Host.

#### **Operation**

1. The Host receives the directional key (36 bits or 128 bits).
2. The Host performs a preliminary directional validation.
3. The Glass Fortress performs the final structural validation.

#### **If both validate the key:**

The Glass Fortress authorizes:

- mutation of the 16 internal bits,
- internal reconstruction (according to the model),
- execution of the path,
- normal/phantom inversion.

#### **If either rejects the key:**

The Glass Fortress:

- blocks all mutation,
- blocks all reconstruction,
- destroys the session,
- erases the private torus,
- blocks the IP.

#### **Absolute directional rule**

The directional key must be validated by **both** the Host **and** the Glass Fortress.

### **3.4. Model A — Validation of the Transmitted Directional Key (36 bits)**

This model is used when developers choose to transmit the directional key over the network.

The Glass Fortress verifies:

#### **1. The 20-bit starting point**

- existing cell,
- directional coherence,
- fractal coherence.

#### **2. The 16-bit transmitted ID**

- existing ID,
- non-corrupted ID,
- non-falsified ID.

#### **3. The 16-bit reconstructed ID**

- internal coherence,
- valid mutation,
- compatibility with the 36-bit model.

#### **4. The 36-bit transmitted directional key**

- fractal coherence,
- absence of drift,
- absence of falsification.

If a single bit is incoherent → **session destroyed**.

### **3.5. Model B — Validation of the Full Directional Key (128 bits)**

This model is used when developers choose not to transmit the key, but to reconstruct it entirely inside the private torus.

The Glass Fortress verifies:

1. The 20-bit starting point
2. The 16-bit transmitted ID

3. The 16-bit reconstructed ID
4. The full 128-bit directional key

The 128-bit directional key is:

- the only key that mutates,
- the only key that inverts,
- the only key that oscillates,
- the only key that moves.

If the directional key is incoherent → **session destroyed**.

### **3.6. Structural Rule: Cell IDs May Repeat**

✓ A cell ID may repeat without any limit. The previous ceiling of 4 cells no longer exists. ✓

An ID may be shared by: → 10 cells, → 100 cells, → 1,000 cells, → 50,000 cells, → or more. ✓

This does not compromise security, because security relies on:

- directional geometry,
- internal mutations,
- directional movements,
- fractal coherence,
- normal/phantom inversion.

✓ 50% of the 16 bits are reserved for directional mutations. The other 50% are active IDs, but their distribution is free and unconstrained.

✓ The 1,000,000 cells of the private torus may therefore:

- massively share the same IDs,
- regroup into directional clusters,
- mutate toward already-used IDs,
- overlap fractally.

This is not a collision → it is a **normal, expected, and validated directional property**.

### **Role of the Glass Fortress**

It validates that:

- ID repetitions are legal,
- mutations toward already-used IDs are coherent,
- the internal fractal remains stable despite clustering,
- no directional drift is detected.

### **3.7. Validation of File Reconstruction**

The Glass Fortress verifies:

- coherence of the directional key (36 bits or 128 bits depending on the model),
- coherence of the torus,
- fractal coherence,
- absence of falsification,
- absence of drift.

It also verifies that:

- the file size is never transmitted,
- the file size is never visible,
- reconstruction is purely directional.

If an incoherence is detected:

- the session is destroyed,
- the private torus is erased,
- the directional key is destroyed,
- reconstruction is cancelled,
- the IP is blocked.

### **3.8. Structural Protection**

The Glass Fortress protects:

✓ the private torus ✓ the directional key of the active model ✓ the file ✓ fractal coherence  
 ✓ directional coherence

It makes any falsification impossible by design.

### 3.9. Automatic Destruction

If an incoherence is detected:

- the session is destroyed,
- the private torus is erased,
- the directional key is destroyed,
- reconstruction is cancelled,
- the IP is blocked.

Destruction is:

- immediate,
- irreversible,
- total.

### 3.10. Summary of Chapter 3

The Glass Fortress:

- validates the directional key according to the chosen model (36 bits or 128 bits),
- validates the transmitted ID (16 bits),
- validates the reconstructed ID (16 bits),
- validates the key jointly with the Host,
- validates file reconstruction,
- accepts repeated IDs without limit,
- protects the private torus,
- destroys the session in case of anomaly.

It is the structural directional firewall, inviolable by design.

## Chapter 4 — Private Torus (Internal Structure, Geometry, Files, Directions, IDs)

*(updated version including the correction for the 36-bit key)*

### 4.1. Overview

The Private Torus is the internal directional structure used to:

- traverse internal directions,
- execute exponential jumps,
- mutate IDs,
- invert normal/phantom,
- reconstruct the file,
- stabilize fractal coherence.

It contains:

- **1,000,000 cells**, each with:
- a 16-bit ID,
- one normal bit,
- one implicit phantom bit,
- four internal directions,
- toroidal displacement,
- circular exponential jump.

The Private Torus is programmed using **two distinct files**, read in parallel.

#### **4.2. Internal Structure of the Torus**

Each cell of the torus contains:

##### **✓ 1. A 16-bit ID**

- 100% of possible values (0 to 65535) are used.
- IDs may repeat without any limit.
- No constraint of uniqueness, collision, or availability.

##### **✓ 2. A normal bit**

- 0 = normal
- 1 = phantom (implicit, never transmitted)

##### **✓ 3. Four internal directions**

Encoded on 2 bits:

#### **Code Direction**

- 00** direction 1
- 01** direction 2
- 10** direction 3
- 11** direction 4

#### **✓ 4. Fractal coherence**

Each cell has internal coherence that:

- updates on every mutation,
- updates on every normal/phantom inversion,
- updates on every directional displacement.

#### **4.3. Cardinal and Diagonal Movements (Toroidal Behavior)**

Internal movements behave like a perfect torus:

✓ Exceed upward → arrival at bottom ✓ Exceed downward → arrival at top ✓ Exceed left → arrival at right ✓ Exceed right → arrival at left

This behavior:

- removes borders,
- removes limits,
- removes dead zones,
- guarantees perfect continuity,
- makes exiting the torus impossible.

✓ Diagonal movements follow the same logic.

#### **4.4. Exponential Jumps (Circular Behavior)**

Each internal direction contains an exponential jump.

This jump:



- does not follow cardinal axes,
- does not follow diagonals,
- does not follow the grid,
- does not follow the torus geometry.

It follows an **internal circular orbit**.

✓ The exponential jump moves along a circle. ✓ When it reaches the end of the circle, it returns to the beginning. ✓ It can never leave the orbit. ✓ It can never block. ✓ It can never drift.

This circular behavior:

- amplifies directional reach,
- creates an independent internal geometry,
- makes paths impossible to reverse,
- stabilizes fractal coherence,
- prevents any external derivation.

#### **4.5. Hybrid Geometry: Torus + Circle**

The internal structure combines two geometries:

✓ **Toroidal geometry** For cardinal and diagonal displacements.

✓ **Circular geometry** For exponential jumps.

This duality:

- creates a living directional surface,
- changes geometry depending on the type of movement,
- makes paths impossible to reverse,
- stabilizes fractal coherence,
- prevents any external derivation.

#### **4.6. Software Architecture of the Private Torus**

The Private Torus is programmed using two distinct files, each with a precise role.

#### **4.6.1. Normal-bit file (pure binary, loaded into RAM)**

This file:

- contains only normal bits,
- is a pure binary file,
- is fully loaded into RAM,
- contains no directions,
- contains no IDs,
- contains no exponential jumps.

Loading into RAM allows:

- instant mutation of cells,
- ultra-fast reading of normal bits,
- continuous file reconstruction,
- immediate normal/phantom inversion.

#### **4.6.2. Directional file (text, stored on disk)**

This file:

- remains on disk,
- is a text file,
- contains the four directions of each cell,
- contains exponential jumps,
- contains the IDs of each cell,
- contains 1,000,000 lines,
- is processed in parallel with the binary file.

**Structure of a line:**

Code

<ID> 101 2^5, 110 3^7, 101 2^7, 111 4^7

#### **4.7. ID Mutation**

Two types of mutation exist, depending on the chosen directional model.

#### 4.7.1. Local Mutation (36-bit model)

*(corrected according to the requested rule)*

In the 36-bit model:

- the transmitted key provides only:
  - the starting cell,
  - the ID of that cell;
- the key does **not** traverse the torus;
- the key performs **no** displacement;
- local mutation is performed **by the execution of the internal directional path**;
- the key never mutates directly;
- no normal/phantom inversion is performed;
- no other cell is modified.

**Local mutation:**

$$ID_{\text{cell}} \leftarrow ID_{\text{result of the path}}$$

Mutation is therefore a **consequence of directional movement**, not a direct action of the key.

#### 4.7.2. Structural Mutation (128-bit model)

In the 128-bit model:

- the internal key traverses the torus,
- inverts normal/phantom,
- reconstructs the file,
- mutates the global torus ID,
- mutates the starting cell ID,
- modifies fractal coherence.

**Structural mutation:**

$$ID_{\text{torus}} \leftarrow ID_{\text{random}}$$

$$ID_{\text{starting cell}} \leftarrow ID_{\text{random}}$$

#### 4.8. Normal / Phantom

Each cell has:

- one normal bit,
- one implicit phantom bit.

✓ The 128-bit model **can** invert normal/phantom. ✗ The 36-bit model **cannot** invert normal/phantom.

#### 4.9. Torus ID

The torus has a 16-bit global ID.

This ID:

- changes with each internal key (128-bit model),
- is chosen randomly over 16 bits,
- may repeat freely,
- has no uniqueness constraint.

#### 4.10. Role of the Torus in File Reconstruction

The Private Torus:

- reconstructs the file bit by bit,
- stabilizes fractal coherence,
- applies directional mutations,
- applies normal/phantom inversions,
- applies exponential jumps,
- applies toroidal displacements.

The file size:

- is never transmitted,
- is never visible,

- is deduced directionally.

#### **4.11. Summary of Chapter 4**

The Private Torus:

- contains 1,000,000 cells,
- each cell has a 16-bit ID,
- IDs may repeat freely and massively,
- cardinal displacements are toroidal,
- exponential jumps are circular,
- hybrid geometry makes paths impossible to reverse,
- is programmed using two files (binary RAM + directional disk),
- the 36-bit model provides the starting cell and its ID,
- local mutation in the 36-bit model is performed by the internal directional path,
- the 128-bit model performs structural mutation,
- normal/phantom inversion depends on the chosen model,
- fractal coherence is continuously updated,
- the hybrid structure makes the torus inviolable.

### **Chapter 5 — ID System (16 bits, Repetition, Mutation, Role)**

*(TorUsNet 7.0 Version — Complete Update)*

#### **5.1. Overview**

The directional protocol uses an identification system (IDs) entirely based on:

- 16-bit IDs,
- 100% of possible values,
- IDs repeatable without limit,
- random mutations,
- local or structural redefinitions,

- an interaction dependent on the chosen directional model:
  - **36-bit model** (local mutation via path),
  - **128-bit model** (structural mutation).

IDs are central to:

- the Private Torus,
- the cells,
- directional mutations,
- fractal coherence,
- file reconstruction.

## 5.2. ID Size: 16 bits

All protocol IDs are 16-bit:

- cell ID,
- torus ID,
- ID transmitted by the 36-bit model,
- ID reconstructed by the 128-bit model,
- ID mutated locally (36-bit model),
- ID mutated structurally (128-bit model).

### ✓ Possible values

$0 \rightarrow 65535$

### ✓ 100% of values are used

No value is reserved. No value is forbidden. No value is set aside.

## 5.3. ID Repetition

IDs may repeat freely, without any limit.

Multiple cells may share the same ID, including:

- tens,
- hundreds,

- thousands,
- or tens of thousands of cells.

#### ✓ **Structural reason**

- The ID is **not** a unique identifier.
- The ID is a **directional marker**.
- Security relies on **directional geometry**, not on ID uniqueness.

#### ✓ **Consequence**

- 1,000,000 cells,
- 65,536 possible IDs,
- normal repetitions,
- normal collisions,
- no impact on fractal coherence.

### **5.4. ID Mutation**

Two types of mutation exist, depending on the chosen directional model.

#### **5.4.1. Local Mutation (36-bit model)**

*(TorUsNet 7.0 update)*

In the 36-bit model:

- the key provides only:
  - the starting cell,
  - the starting ID;
- the key does **not** traverse the torus;
- the key performs **no** displacement;
- local mutation is performed **by the execution of the internal directional path**;
- no other cell is modified;
- no normal/phantom inversion is performed.

**Local mutation:**

$$ID_{\text{cell}} \leftarrow ID_{\text{result of the path}}$$

Mutation is therefore a **consequence of directional movement**, not a direct action of the key.

#### 5.4.2. Structural Mutation (128-bit model)

In the 128-bit model:

- the internal key traverses the torus,
- inverts normal/phantom,
- reconstructs the file,
- mutates the global torus ID,
- mutates the starting cell ID,
- modifies fractal coherence.

**Structural mutation:**

$$ID_{\text{torus}} \leftarrow ID_{\text{random}}$$

$$ID_{\text{starting cell}} \leftarrow ID_{\text{random}}$$

Random IDs are chosen over 16 bits, even if they are already used.

#### 5.5. Random Mutation on 16 Bits

During a mutation, the program chooses an ID at random over 16 bits, even if that ID is already used.

#### ✓ Properties

- mutation = random draw from 0–65535,
- no uniqueness constraint,
- no collision constraint,
- no availability constraint,
- purely directional mutation.

#### 5.6. Role of IDs in the Private Torus

Each torus cell has:



- a 16-bit ID,
- one normal bit,
- one implicit phantom bit,
- four internal directions,
- toroidal displacement,
- circular exponential jump.

The ID:

- identifies the cell in the directional file,
- serves as an entry point for mutations,
- serves as a marker for fractal coherence,
- serves as a reference for internal reconstruction.

### **5.7. Torus ID**

The torus has a 16-bit global ID.

This ID:

- changes with each internal key (128-bit model),
- is chosen randomly,
- may repeat,
- has no uniqueness constraint.

The 36-bit model never modifies the global torus ID.

### **5.8. Interaction Between IDs and Directional Models**

#### **✓ 36-bit model**

- provides the starting cell,
- provides the starting ID,
- triggers an internal path,
- local mutation is performed by the path,
- never modifies the torus ID,

- never inverts normal/phantom.

#### ✓ 128-bit model

- traverses the torus,
- inverts normal/phantom,
- reconstructs the file,
- mutates the torus ID,
- mutates the starting cell ID,
- stabilizes fractal coherence.

### 5.9. Fractal Coherence and IDs

Fractal coherence:

- accepts repeated IDs,
- accepts collisions,
- accepts random mutations,
- stabilizes automatically,
- updates with each directional mutation.

IDs are not a security structure. They are a **directional structure**.

### 5.10. Summary of Chapter 5

The ID system works as follows:

- all IDs are 16-bit,
- 100% of values are used,
- IDs may repeat freely,
- mutations choose a random ID,
- the 36-bit model provides the starting cell and mutates via the internal path,
- the 128-bit model mutates the torus and the starting cell,
- ID collisions are normal,
- fractal coherence remains stable,

- security relies on directional geometry, not on ID uniqueness.

## **Chapter 6 — Directional Handshake (Host ↔ Client Sequence)**

*(TorUsNet 7.0 Version — Complete Update)*

### **6.1. Overview**

The directional handshake is the structural mechanism that establishes a coherent session between a Host and a Client.

Unlike classical protocols (TLS, SSH, IPSec) relying on:

- private keys,
- certificates,
- signatures,
- mathematical derivations,
- cryptographic exchanges,

the directional handshake relies exclusively on:

- the internal geometry of the tori,
- fractal coherence,
- directional movements,
- internal IDs,
- joint validation by Host + Glass Fortress.

It contains no cryptography, no derivation, no exploitable mathematical structure.

### **6.2. Objectives of the Handshake**

The directional handshake ensures:

1. validation of the first directional key (36 bits or 128 bits),
2. initial synchronization between Host and Client,
3. registration of the Client's IP as a trusted IP,
4. creation of the global session ID (16 bits),

5. activation of the Glass Fortress,
6. initialization of the Private Torus,
7. establishment of the torus global ID (16 bits),
8. full directional synchronization through 5 transmissions.

These steps are structural, not cryptographic.

### **6.3. The Official Sequence: 5 Transmissions**

The directional sequence consists of **5 transmissions**, each with a precise structural role. It is mandatory, inviolable, and forms the foundation of directional synchronization.

#### **Transmission 1 — Host → Client**

##### **Initial sending of the directional key**

The Host sends a directional key:

- **36-bit model:**
  - 20 bits: starting cell,
  - 16 bits: starting ID.
- **128-bit model:**
  - 128 bits: full directional key.

The Client:

- receives the key,
- verifies its structure,
- verifies its model (36 or 128 bits),
- verifies the targeted ID,
- verifies initial fractal coherence,
- sends the key to the Glass Fortress for joint validation.

No directional execution is performed yet.

#### **Transmission 2 — Client → Host**

##### **Return of the key to confirm reception**

The Client returns **the exact same key** to the Host.

The Host:

- compares the sent key with the received key,
- validates reception,
- registers the Client's IP as trusted,
- activates the Glass Fortress.

If the key differs → **immediate destruction**.

### **Transmission 3 — Host → Client**

#### **Execution authorization**

The Host sends a second copy of the key.

The Client:

- compares the returned key with the received key,
- validates coherence,
- executes the internal directional path in its Private Torus,
- updates internal paths,
- updates fractal coherence.

#### **Effects depending on the model:**

##### **✓ 36-bit model**

- local mutation of the starting cell ID,
- mutation performed by the path,
- no normal/phantom inversion.

##### **✓ 128-bit model**

- normal/phantom inversion,
- structural mutation of the torus ID,
- structural mutation of the starting cell ID.

The Client's Private Torus becomes:

- living,
- oscillating,
- modified.

#### **Transmission 4 — Client → Host**

##### **Confirmation of path execution**

The Client returns the key after executing the internal path.

This indicates to the Host:

- that local (36-bit) or structural (128-bit) mutation has been performed,
- that internal paths have been updated,
- that fractal progression is coherent,
- that the Client's Private Torus is up to date.

The Host may then:

- execute the same path,
- apply the same mutations,
- synchronize its Private Torus with the Client's.

#### **Transmission 5 — Host → Client**

##### **Final synchronization confirmation**

The Host returns the key one last time to confirm:

- that it executed the path,
- that it applied the mutations,
- that its Private Torus is synchronized with the Client's,
- that the session is directionally stable.

The Client receives this confirmation and the session becomes:

- coherent,
- stable,
- inviolable.

## **6.4. Why Five Transmissions?**

### **6.4.1. Complete bidirectional validation**

Host validates Client. Client validates Host. Then each validates the other's directional execution.

### **6.4.2. Protection against injections**

An attacker cannot:

- intercept the key,
- modify it,
- resend it,
- falsify it.

Any incoherence is structurally detected.

### **6.4.3. Protection against repetitions**

A repeated key:

- is detected,
- is rejected,
- destroys the session.

### **6.4.4. Protection against derivations**

An attacker cannot derive:

- the starting cell,
- the starting ID,
- internal movements,
- mutations,
- fractal coherence.

The directional structure is non-derivable.

### **6.4.5. Perfect directional synchronization**

The 5 transmissions guarantee that:

- both Private Tori executed the same path,
- mutations are identical,
- internal geometry is coherent,
- no divergence is possible.

### **6.5. Initial Synchronization**

Once the key is validated:

- the Authentication Torus is used only once,
- the Private Torus is initialized,
- the global ID is created,
- normal/phantom pairs are defined (128-bit model),
- internal paths are activated,
- the torus global ID is established,
- the Glass Fortress is armed.

This synchronization is fractal, not mathematical.

### **6.6. Initial Reconstruction of the Private Torus**

The first key triggers:

- directional initialization,
- setup of internal paths,
- local (36-bit) or structural (128-bit) mutation,
- fractal stabilization,
- activation of internal directions,
- activation of exponential jumps.

The Private Torus becomes:

- living,
- oscillating,
- operational.



## **6.7. Error Handling During the Handshake**

### **6.7.1. Incoherent key**

If the key:

- contains an invalid ID,
- contains an impossible cell,
- contains an illegitimate model, → the session is destroyed.

### **6.7.2. Modified key**

If the returned key differs:

- immediate destruction,
- Private Torus erased,
- IP invalidated.

### **6.7.3. No response**

If the Host receives no response after 5 seconds:

- the key is considered lost,
- a new key is generated,
- with a different starting cell,
- but reconstructing the same directional structure.

No retransmission of the same packet is performed.

## **6.8. Summary of Chapter 6**

The directional handshake:

- relies on 5 transmissions,
- validates the first directional key,
- registers the Client's IP as trusted,
- initializes the Private Torus,
- activates the Glass Fortress,
- creates the global ID,

- establishes the unique torus ID,
- guarantees total directional coherence,
- protects against injections, repetitions, derivations, and falsifications.

It forms the foundation of structural directional security.

## **Chapter 7 — Multi-Torus and Parallel Networks**

*(TorUsNet 7.0 Version — Complete Update)*

### **7.1. Overview**

The multi-torus system is one of the most powerful foundations of the directional architecture. It allows the creation of multiple independent directional networks, each possessing:

- its own Private Torus,
- its own Authentication Torus,
- its own directional keys (36-bit or 128-bit),
- its own internal IDs,
- its own global session ID,
- its own global torus ID,
- its own Glass Fortress.

The Public Torus, pre-generated and universal, serves as the common directional base for all branches.

Each parallel network is completely isolated from the others, even if they coexist:

- on the same machine,
- on the same system,
- in the same network environment.

The multi-torus system enables:

- directional segmentation,
- structural sovereignty,

- total branch isolation,
- creation of inviolable private networks,
- controlled directional duplication.

## **7.2. Definition of a Parallel Network**

A parallel network is a complete instance of the directional architecture composed of:

1. The Public Torus (pre-generated, universal)
2. An Authentication Torus
3. A Private Torus
4. A set of directional keys
5. A Glass Fortress
6. A global session ID
7. A global torus ID

Each parallel network is an independent directional universe.

It shares:

- no keys,
- no IDs,
- no Private Torus,
- no fractal coherence,
- no internal paths.

## **7.3. Creation of a New Directional Branch**

Creating a new branch follows three steps.

### **7.3.1. Use of the Pre-Generated Public Torus**

The Public Torus:

- is pre-generated,
- is provided with the software,
- is universal,

- is identical for all installations,
- is non-modifiable,
- is non-derivable.

It serves as the common directional base but contains no private information.

### **7.3.2. Generation of the Authentication Torus**

An Authentication Torus is generated. It validates the first directional key and initializes:

- the first normal/phantom pairs (128-bit model),
- the first internal paths,
- the global session ID,
- the global torus ID.

This torus is used only once: during the initial handshake.

### **7.3.3. Generation of the Private Torus**

A Private Torus is generated. It will be reconstructed fractally from directional keys:

- normal/phantom inversion (128-bit model),
- ID mutation (local or structural),
- activation of internal paths,
- fractal stabilization.

Each Private Torus is independent from all other existing Private Tori.

## **7.4. Directional Branches**

Each parallel network is called a directional branch.

A branch possesses:

- its own Private Torus,
- its own fractal progression,
- its own internal IDs,
- its own Glass Fortress,
- its own global ID,

- its own global torus ID.

#### **7.4.1. Independent Branches**

Branches do not communicate with each other. They cannot:

- share keys,
- share IDs,
- share Private Tori,
- share internal paths,
- share mutations.

#### **7.4.2. Sovereign Branches**

Each branch is sovereign:

- it does not depend on any other branch,
- it cannot be influenced by another branch,
- it cannot be derived by another branch.

### **7.5. Directional Duplication (Offline Primary Torus)**

A key feature of the multi-torus system is offline directional duplication.

#### **7.5.1. Primary Torus**

A primary torus may be:

- generated,
- stored offline,
- transferred physically,
- used to create a new branch.

#### **7.5.2. Offline Transfer**

The primary torus may be transferred:

- on a USB drive,
- on a hard disk,
- on physical media,

- on any non-network medium.

This transfer contains no directional keys. It contains only:

- directional geometry,
- internal IDs,
- internal directions,
- exponential jumps.

### **7.5.3. Creation of a Branch from the Primary Torus**

Once the primary torus is transferred:

- a new branch is created,
- a new Private Torus is reconstructed,
- a new global ID is generated,
- a new global torus ID is established,
- a new Glass Fortress is activated.

The branch is completely independent from the original branch.

## **7.6. Structural Isolation**

Structural isolation is total.

### **7.6.1. Fractal Isolation**

Fractal actions of one branch:

- cannot influence another branch,
- cannot derive another branch,
- cannot reconstruct another branch.

### **7.6.2. Directional Isolation**

Internal paths of one branch:

- cannot be used in another branch,
- cannot be derived by another branch.

### **7.6.3. ID Isolation**

Internal IDs:

- are not compatible between branches,
- cannot be transferred,
- cannot be derived.

### **7.7. Multi-Torus and Post-Quantum Security**

The multi-torus system strengthens post-quantum security:

A quantum computer cannot derive:

- a Private Torus,
- a branch,
- a directional key,
- an ID,
- the global torus ID.

Even with infinite power, an attacker cannot:

- explore a Private Torus,
- derive a Private Torus,
- reconstruct a Private Torus,
- analyze a Private Torus.

Private Tori are geometrically inviolable.

### **7.8. Multi-Torus and Photonic Security**

With the arrival of photonic computers:

- mathematical structures become obsolete,
- classical cryptography becomes vulnerable.

The multi-torus system is designed for this era:

- Private Tori are non-transmissible,
- non-derivable,
- non-analyzable,

- non-explorable.

A photonic computer cannot:

- penetrate a Private Torus,
- analyze a Private Torus,
- derive a Private Torus.

## **7.9. Summary of Chapter 7**

The multi-torus system enables:

- creation of parallel networks,
- total branch isolation,
- offline directional duplication,
- structural sovereignty,
- post-quantum security,
- photonic security.

Each branch is an independent directional universe, inviolable by design. The Public Torus, pre-generated and universal, serves as the common directional base — but never as a shared Private Torus.

## **Chapter 8 — Directional Ports and Internal Communication Geometry**

*(TorUsNet 7.0 Version — Complete Update)*

### **8.1. Overview**

The directional ports of TorusNet are a structural extension of classical ports. Unlike TCP/UDP ports (0–65535), TorusNet uses:

- extended ports,
- several billions of possibilities,
- mutable ports,
- non-scannable ports,
- non-derivable ports,



- non-falsifiable ports,
- ports structurally validated by the Glass Fortress.

A directional port is not:

- an integer,
- a socket,
- a TCP channel,
- a network identifier.

A directional port is an internal structure of the Private Torus, generated directionally, mutated at each torus renewal and at each new connection.

## **8.2. Definition of a Directional Port**

A directional port is:

- an extended port number (several billions of possibilities),
- generated by the Private Torus,
- mutated at each torus renewal,
- mutated at each new connection,
- impossible to predict,
- impossible to scan,
- impossible to derive.

### **Structural rule: one connection per port**

Each directional port:

- allows only one connection,
- is invalidated as soon as the session ends,
- is replaced by a new extended port at the next connection.

## **8.3. Why Directional Ports Are Inviolable**

### **8.3.1. Absence in the Classical Network Layer**

An attacker cannot:

- scan ports,
- probe ports,
- open a port,
- send a packet to a port.

Directional ports do not exist in the classical network layer. They exist only inside the internal mechanics of the Private Torus.

### **8.3.2. Mutable and Non-Derivable Ports**

Directional ports:

- mutate automatically,
- are never reused,
- cannot be predicted,
- cannot be mathematically derived.

### **8.3.3. Validation by the Glass Fortress**

The Glass Fortress:

- monitors directional ports,
- validates their coherence,
- destroys the session if a port is violated,
- blocks the hostile IP in the operating system firewall.

### **8.3.4. Non-Transmissible Ports**

A directional port:

- cannot be sent,
- cannot be copied,
- cannot be exported,
- cannot be derived.

It exists only locally, inside the Private Torus.

## **8.4. Mutation of Directional Ports**

#### 8.4.1. Mutation During Torus Renewal

Each time the Private Torus is regenerated:

- a new set of extended ports is generated,
- all previous ports become invalid.

#### 8.4.2. Mutation at Each New Connection

At each new session:

- a new extended port is generated,
- only one connection is allowed on this port,
- any attempt at a second connection triggers immediate destruction.

#### 8.5. Directional Ports and Transmission

**Host → Client** The Host opens an extended directional port. The Client must respond on the same port.

**Client → Host** The Client returns the directional key on this port. The Host validates coherence.

**Host → Client** The Host returns the final key on the same port.

**Port violation** If a directional port is violated:

- the session is destroyed,
- the Private Torus is erased,
- the IP is blocked.

#### 8.6. Directional Ports and Multi-Torus

In a multi-torus environment:

- each branch has its own directional ports,
- ports are not compatible between branches,
- ports cannot be derived from one branch to another.

A directional port is strictly local to its branch.

#### 8.7. Directional Ports and Post-Quantum Security

Directional ports are inviolable by a quantum computer:

- impossible to derive an extended port,
- impossible to predict a mutation,
- impossible to explore the Private Torus,
- impossible to reconstruct internal ports.

## **8.8. Directional Ports and Photonic Security**

Photonic computers can break:

- RSA,
- ECC,
- TLS,
- SSH.

But they cannot:

- analyze directional geometry,
- derive a torus,
- reconstruct a directional port,
- predict a mutated extended port.

Directional ports are structurally inviolable.

## **8.9. Summary of Chapter 8**

Directional ports:

- are classical ports extended to several billions of possibilities,
- mutate at each torus renewal,
- mutate at each new connection,
- allow only one connection per port,
- are non-scannable,
- non-derivable,
- non-falsifiable,
- validated by the Glass Fortress,

- inviolable by design.

They form the internal communication layer of TorusNet, entirely independent from classical network protocols.

## **Chapter 9 — DNS and HTTPS Authentication (Temporary Trust Layer)**

*(TorUsNet 7.0 Version — Complete Update)*

### **9.1. Overview**

Before directional geometry takes full control of the session, a temporary trust layer is required to:

- identify the requested Host,
- avoid malicious redirections,
- prevent classical MITM attacks,
- secure the transmission of the first directional key,
- ensure that the Client is contacting the correct Host.

This layer relies on two classical Internet technologies:

- **DNS** (domain name resolution),
- **HTTPS** (TLS certificate).

These technologies are not part of the directional mechanics. They serve only as an entry buffer before activation of:

- the Authentication Torus,
- the Private Torus,
- the Glass Fortress,
- the internal IDs,
- the global ID,
- the global torus ID.

Once the first key is validated, DNS and HTTPS disappear completely.

### **9.2. Role of DNS in TorusNet**

DNS is used for one single purpose:

**Identify the requested Host before entering directional geometry.**

### **9.2.1. Domain Verification**

When a Client contacts a Host:

1. DNS resolves the domain name.
2. The protocol verifies that this domain belongs to the list of trusted Hosts.
3. If the domain is unknown or suspicious:
  - the connection switches to user-confirmation mode,
  - no directional key is transmitted.

### **9.2.2. Protection Against Redirections**

DNS protects against:

- malicious redirections,
- fake domains,
- cloned Hosts,
- traffic hijacking.

### **9.2.3. Strictly Temporary Role**

Once the first key is validated:

**DNS is no longer used. Directional geometry takes over.**

## **9.3. Role of HTTPS in TorusNet**

HTTPS is used to:

**Secure the transmission of the first directional key.**

### **9.3.1. Certificate Verification**

The Client verifies:

- the TLS certificate,
- the validity of the domain,
- the absence of classical MITM interception.

If the site does not support HTTPS:

- the connection switches to user-confirmation mode,
- no directional key is sent.

### **9.3.2. Protection Against Classical Attacks**

HTTPS protects against:

- interceptions,
- injections,
- traffic manipulation,
- downgrade attacks.

### **9.3.3. Strictly Temporary Role**

Once the first key is transmitted:

**HTTPS disappears. The session becomes 100% directional.**

## **9.4. Interaction DNS ↔ HTTPS ↔ Authentication Torus**

The complete sequence is:

1. DNS identifies the Host.
2. HTTPS secures the first transmission.
3. The Authentication Torus validates the first key.
4. The Glass Fortress activates the directional session.
5. The Private Torus takes control.
6. DNS and HTTPS are abandoned.

### **9.4.1. Full Trust Case**

- DNS valid
- HTTPS valid

→ The first key is transmitted automatically.

### **9.4.2. Partial Trust Case**

- DNS valid

- HTTPS absent

→ User confirmation required.

#### **9.4.3. Suspicion Case**

- DNS suspicious
- HTTPS absent or invalid

→ Connection blocked until explicit validation.

#### **9.5. Why TorusNet Still Uses DNS and HTTPS**

TorusNet is a post-Internet directional architecture. However, the current ecosystem requires:

- the use of domain names,
- the use of TLS certificates,
- the use of HTTP(S) servers.

DNS and HTTPS therefore act as a bridge between:

- classical Internet,
- directional Internet.

##### **9.5.1. Final Objective: Total Removal**

In future versions:

- DNS will be replaced by directional DNS,
- HTTPS will be replaced by native directional validation,
- certificates will be replaced by directional IDs,
- connections will be fully geometric.

#### **9.6. Interaction with Directional Updates**

##### **ID Inversion and Mutation**

Once the first key is validated:

- the ID of the starting case mutates,
- the global ID is created,



- DNS and HTTPS are no longer needed.

### **Autonomous Directional Session**

The session becomes:

- continuous,
- alive,
- autonomous.

## **9.7. Summary of Chapter 9**

This chapter establishes that:

- DNS and HTTPS serve only to secure the first key,
- they disappear immediately after validation,
- the session becomes 100% directional,
- the Glass Fortress takes control,
- the Private Torus becomes the sole communication layer,
- internal IDs replace certificates,
- directional ports replace sockets,
- directional geometry replaces cryptography.

### **Update — Removal of Authentication via MAC Address, CPU ID, and Motherboard ID**

*(TorusNet 7.0 Version — Structural Update)*

#### **Context of the Modification**

The early experimental versions of TorusNet used hardware-based authentication relying on:

- the MAC address,
- the processor ID,
- the motherboard ID.

This approach had been considered to strengthen the initial validation of the Host and Client before activation of the Authentication Torus.

However, this method was abandoned in TorUsNet 7.0 for structural, technical, and legal reasons.

### **1. Primary Reason: Confidential Nature of Hardware Data**

Hardware identifiers (MAC, CPU ID, Motherboard ID) are considered:

- sensitive data,
- personal information,
- confidential elements,
- persistent markers incompatible with the philosophy of Tor and TorusNet.

### **TorusNet cannot depend on private information.**

The directional architecture relies on:

- internal geometry,
- private tori,
- directional IDs,
- fractal coherence,
- the Glass Fortress.

It must never rely on persistent hardware identifiers.

### **2. Compliance with Tor's Fundamental Principles**

The Tor network imposes an essential rule:

#### **No hardware data must be used to identify a user.**

TorusNet, as a post-Tor directional architecture, must respect this foundation:

- no MAC address,
- no CPU ID,
- no motherboard ID,
- no persistent hardware identifier.

These data types are incompatible with:

- structural anonymity,

- non-traceability,
- non-correlation,
- directional sovereignty.

### **3. Technical Reason: Incompatibility with Directional Geometry**

Hardware identifiers:

- are not directional,
- are not fractal,
- are not mutable,
- cannot be integrated into a torus,
- cannot be validated by the Glass Fortress.

They introduce a static structure, whereas TorusNet relies on:

- mutable IDs,
- mutable directional ports,
- living tori,
- internal paths,
- fractal mutations.

A hardware identifier cannot follow directional logic.

### **4. Security Reason: Impossible to Guarantee Hardware Integrity**

Hardware identifiers can be:

- falsified,
- cloned,
- virtualized,
- masked,
- modified by the operating system.

They do not constitute a reliable foundation for a directional architecture.

TorusNet 7.0 requires validation that is:

- internal,
- geometric,
- fractal,
- non-transmissible,
- non-derivable.

Hardware identifiers meet none of these requirements.

## **5. Complete Replacement by Directional Validation**

In TorUsNet 7.0, hardware authentication is replaced by:

### **✓ The Authentication Torus**

Validates the first directional key.

### **✓ The Glass Fortress**

Validates the structural coherence of the Host.

### **✓ Directional IDs**

Mutable, non-derivable, non-transmissible.

### **✓ Directional Ports**

Mutable, non-scannable, non-falsifiable.

### **✓ The 5-Transmission Directional Handshake**

Complete Host  $\leftrightarrow$  Client synchronization.

**No hardware data is used.**

## **6. Conclusion of the Update**

Authentication via:

- MAC address,
- CPU ID,
- motherboard ID

has been **permanently removed** in TorUsNet 7.0.

This decision guarantees:

- compliance with Tor principles,
- structural confidentiality,
- non-traceability,
- directional sovereignty,
- fractal coherence,
- post-quantum and photonic security.

TorUsNet 7.0 now relies exclusively on directional geometry and the internal mechanisms of the torus.